



University of Cape Town

Department of Information Systems

INF3003W – Coding Test

April 2026

Instructions to Candidates

1. This assessment will take place **23rd April 2026 from 10:00 to 13:00 PM** in the BSG/THATHU Labs or Online. This includes reading time.
2. If you are expecting any challenges in this regard, please inform the course administrator in advance.
3. This test is an open book. However, you are **not to communicate with other candidates during the assessment**. You may not help another student to engage in dishonest conduct and may not allow or assist another student to copy. Any dishonesty will render you liable to disqualification and to disciplinary action.
4. For this test you need to create a project in Visual Studio Code and submit the final complete solution on Github.
5. Implement all the required features as part of the solution.
6. The final solution will be marked out of 50 and will contribute 1% towards your total marks. All the required systems features will be marked, even if the program does not run without errors.
7. You must submit your solution to the **Mock-Test** Github repository before 13H00 PM.
8. *For Marking Purposes*

Q1	Q2	Q3	Q4	Q5	Q6	TOTAL
/5	/5	/5	/5	/20	/10	/50

CASE STUDY – Pedal Power Association (PPA)

© University of Cape Town, All rights reserved

Note :These materials are for educational purposes and for your personal use and may not be shared, distributed, sold or placed on any third party websites without the permission of the University.

Most of the information below is obtained from the PPA website: <http://www.pedalpower.co.za/>; Those sections marked in Italics were taken from the website unchanged.

Background

Founded in 1976, the PPA is a Public Benefit Organisation committed to promoting cycling and the interests of cyclists. Recreational cycling (on- and off-road), development through cycling, safe cycling and advocacy are among the PPA's priorities.

The Pedal Power Association (PPA) is the largest cycling organisation in South Africa. It currently has some 13 000 paid-up members, spread all over the country, although the largest component is in the Western Cape, where the Association has its headquarters. The Pedal Power Association came into being as a result of the running of the first Cycle Tour that was to develop into the now world famous Cape Town Cycle Tour (previously the 'Cape Argus Pick 'n Pay Cycle Tour').

The main purpose and object of the Association is to promote cycling and the interests of cyclists. The Association has the following ancillary objectives:

- To be a representative body for the furtherance of the interests of cyclists.*
- To promote cycling as a recreational activity, a sport and as a means of transportation.*
- To improve conditions for cyclists with particular regard to their safety.*
- To arrange and organise cycle tours, fun rides and outings.*
- To co-ordinate cycle tours, fun rides and outings organised by other bodies and to assist them in their efforts.*
- To establish contact and liaise with and where appropriate affiliate with other organisations having similar interests and objectives.*

An elected committee of 12 persons, who meet once per month, governs the Association. The PPA Office is staffed by fourteen employees, who administer the day-to-day running of the Association and its members, provide guidance to event organisers on the extensive PPA funride calendar, organise the PPA-own events, and run the various PPA campaigns like the COOL KID ON A BIKE, SAFE CYCLING and Bike4All campaigns.

Events on the PPA calendar are split between (a) events owned and organised by the Association (including the One Tonner, Tietoria Elgin Valley MTB, Maluti Double90, The MTB Challenge, Medihelp Stellenbosch Cycle Tour, Tour de PPA, Mangaung Cycle Tour and various social rides and tours), and (b) events that are owned and organised by various service organisations and clubs as fundraisers for a charitable or cycling-related cause.

The Association's income stems from membership fees and from a share in the profit of the Cape Town Cycle Tour. Income from membership fees are used to run the Association.

From the above excerpt it is apparent that PPA, in addition to maintaining its member base, is also involved in several events.

PPA Members

The Pedal Power Association recognised the impact that COVID-19 has had on budgets of current and potential members. They are also looking to attract many new cyclists and hence decided to reduce the membership fees to R100 per person for the 2021/2022 membership year.

This membership is critical for them to be able to support cycling and the interests of cyclists. These contributions help fund their activities, as well as giving PPA credibility as an organisation representing thousands of cyclists.

Amongst others, a member will have the following benefits:

- Discounted entries to all PPA own events (once events resume)
- A free annual seeded funride number (road)
- A guaranteed entry to the 2023 Cape Town Cycle Tour with preferential entry time
- Newsletters to stay up to date with events and the world of cycling
- Preferential seeding at funrides
- Several discounts and membership benefits from our external partners, as listed on their website which will be updated regularly!

PPA Rides

PPA coordinates several events throughout the year and they also assist other bodies in their organisation of similar events. The events included in the yearly calendar of PPA are funrides, mountain bike rides (MTB-rides), Social rides, and PPA league rides for the more competitive cyclists. Members also receive a calendar of events and funride information throughout the year.

A company called RaceTec, times PPA rides. Each PPA member needs to be in possession of a timing chip and will receive a time for each ride. These times are then uploaded to a race results database which resides with a provider and members can look up their ride results on the PPA website.

Members can update their memberships from the PPA website.

PPA Event System

As a service organisation PPA is committed to keep their systems current, providing PPA members with up to date member information and details of the rides on their yearly calendar.

The immediate need is to ensure that event information is managed effectively and you are tasked to re-design the event management system for PPA. This system should integrate seamlessly with the existing member information.

Membership Management

Licensed riders (**Members**) need to have a member number (ID), ASPNetUserID, RaceTechID, FirstName, Surname, email, MemberType, JoinDate and Current Subscription (PaidUp).

Members can take part in **PPA league events** in different **categories** depending on their age (elites; sub veterans; veterans; masters; junior scholars; senior scholars and ladies). League events have specific rules and riders also receive prizes after each race.

Members need to be able to register on the system, with the default member status being non-paid up. A paid-up member can register for any event (race entry). A ride_entry is linked to a member. A member can view his ride entry, but is not able to edit it. An administrator is able to edit the ride entries.

PPA Administrators need to be able to edit members subscriptions to indicate if they are paid-up. (In future this will be automated by linking to a SnapScan system).

Ride Management (Events)

Most **Rides** have some common attributes. Every ride has an RideID, ride name, a ride description, ride date, ride distance and an ride type. The ride type can be one of the following (funrides, mountain bike rides (MTB-rides), Social rides, and League rides).

Date:	Type:	Event Name:	Region:	Entries Close:	More Info / Enter:
Saturday, 27 August 2022		Old Hermon Gravel Ride	Western Cape	Wednesday, 17 August 2022	More Info
Sunday, 28 August 2022		Entelect Race for Victory 2022	Gauteng	Thursday, 18 August 2022	More Info
Saturday, 03 September 2022		Vines & Views MTB	Western Cape		More Info
Saturday, 10 September 2022		aQuellé Tour Durban 2022 MTB	Kwazulu-Natal	Thursday, 01 September 2022	More Info
Sunday, 11 September 2022		aQuellé Tour Durban 2022 Road	Kwazulu-Natal	Thursday, 01 September 2022	More Info
Saturday, 17 September 2022		Blue Diamond Classic 2022	Gauteng	Monday, 05 September 2022	More Info
Sunday, 18 September 2022		Boland Gravel Monster 2022	Western Cape	Monday, 05 September 2022	More Info
Sunday, 25 September 2022		2022 Lions Karoo to Coast MTB Challenge	Western Cape	Thursday, 01 September 2022	More Info
Sunday, 02 October 2022		Boland Race of Truth	Western Cape		More Info
Thursday, 06 October 2022		Tour du Nord 4 Stage Road Cycling Race 2022	Gauteng	Sunday, 18 September 2022	More Info
Friday, 07 October 2022		Val de Vie MTB Expedition 2022	Western Cape	Sunday, 18 September 2022	More Info
Sunday, 09 October 2022		West Coast Express	Western Cape	Friday, 07 October 2022	More Info

Administrators need to be able to create a new ride, edit a ride but not delete a ride.

Ride Entries

A paid-up member can enter any event (ride entry). A ride entry is linked to a member and a ride and records the member ID, ride id and members time for the particular ride.

Solution Requirements:

For this test you will create an ASP.NET Core 10.0 Razor web application.

The application will allow a PPA Administrator to create and edit members, events and rides and members to enter for events and view their ride times. The following steps should allow you to develop the entire solution.

<<<To assist you with the mock test, some instructions are provided **in bold**. This will not be provided in the actual test/exam >>>>

Create the Project Setup[0 Marks]

Create a new C# ASP.NET Core Web App (Razor pages) project in Visual Studio Code.

1. Check out the starter solution from Github repository (Link is in the Assignment).
2. The target Framework is ASP .NET Core 10.0 Web App (C#) with Razor Pages.
3. The authentication Type is “**Individual Accounts**”.

Open VS CODE, navigate to your desired folder and type:

```
dotnet new webapp -n Your_Student_ID --framework net10.0 --auth Individual
```

Change to your new project folder and then type `code .` or `code-insiders .` to open the project.

Question 1 – Scaffolding Identity [5 Marks]

You need to scaffold the Account Register, Login, LogOut, RegisterConfirmation and Manage/Index pages for Identity: (5)

1. Install the packages that you need for the project.

*<<<<<<<<<<<Some of these packages may already be installed or might not be required
in your solution and are provided as an example>>>*

```
dotnet add package
Microsoft.VisualStudio.Web.CodeGeneration.Design
dotnet add package
Microsoft.AspNetCore.Diagnostics.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet tool install --global dotnet-ef
dotnet tool install --global dotnet-aspnet-codegenerator
```

2. Scaffold the Identity pages.
`dotnet aspnet-codegenerator identity -dc ApplicationDbContext`
3. Disable default account verification.

In Program.cs, use the following

```
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
    options.SignIn.RequireConfirmedAccount = false)
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

4. Create the migration and update the database.
dotnet ef migrations add InitIdentity

dotnet ef database update

5. Test identity.

dotnet run

Question 2 – Create the Member Class [5 Marks]

Create a new public class called Models/Member.cs (in the Models folder). The class should contain the following items and extend IdentityUser:

1. Create all the following properties with public get and set accessors. Choose the most efficient data type for each property. In addition to the properties, your Rides class should also cater for nullable fields and display friendly names and the correct field lengths. See the table below: (5)

Property	Description	Key	Data Type	Field Length	Nullable
MemberID	The primary key	PK	Int		No
AspNetUserID	Link to ASP User ID		String		No
RaceTechID	Member's Racetech Chip ID		String		No
FirstName	The name of the Member		String		No
LastName	The surname of the member		String		No
MemberEmail	The email of the logged-in member.		String		
MemberType	The type of Member based on the enum	Enum	Int		No
JoinDate	The Member Joining date		Datetime		No
PaidUp	Flag to check if member is paid up. Default is False		Boolean		No

Table 1 Member Class

<<<<The following example is provided to assist you in the mock-test and will not be available in the following classes or the test/exam>>>>

Models represent the data entities in your application. Let's create a the Member model.

In VS Code, create a Models folder in the project root (right-click the project > New Folder > Models).

Inside Models, create a Member.cs file with the following code:

```
namespace STDNUM001.Models;

public class Member : IdentityUser
{
    // Primary key (EF Core auto-generates Id as identity column)
```

```

    public int MemberId { get; set; }
    public string? AspNetUserID { get; set; }
    public string RaceTechID { get; set; } = string.Empty;
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string MemberEmail { get; set; } = string.Empty;
    public MemberType MemberType { get; set; }
    public DateOnly JoinDate { get; set; }
    public Boolean PaidUp { get; set; }
}

```

Here, MemberId is the primary key (EF Core conventions assume Id or MemberId as the primary key).

The Name field is required (non-nullable), while Description is optional (nullable via ?).

2. Create a public enumeration called MemberType which represents the type of member. Array of Elites, SubVeterans, Veterans, Masters, JuniorScholars, SeniorScholars, Ladies) [Required]

```

public enum MemberType
{
    Elites, SubVeterans, Veterans, Masters, JuniorScholars,
    SeniorScholars, Ladies
}

```

To eliminate the warnings from nullable reference types and to scaffold the CRUD, remove the line `<Nullable>enable</Nullable>` from the Your_Student_ID.csproj file.

3. Remember to add `using Microsoft.AspNetCore.Identity;` to your class.
4. Run migrations and update the database.

Question 3 – Create the Ride Class [5 Marks]

Create a new public class called Models/Ride.cs (in the Models folder)-which represents all future Rides. The class should contain the following items:

1. Create all the following properties with public get and set accessors. Choose the most efficient data type for each property. In addition to the properties, your Rides class should also cater for nullable fields and display friendly names and the correct field lengths. See the table below: (3)

Property	Description	Key	Data Type	Field Length	Nullable
RideID	The primary key	PK	Guid		No
RideName	The name of the Ride		NVarchar	25	No
RideType	The type of Ride based on the enum	Enum	Int		No
RideDescription	A short description of the Ride		String		No
RideDate	The Ride date		Datetime		No
RideDistance	The distance of the ride		Decimal	18,2	No

Table 2 Ride Class

2. Create a public enumeration called RideType which represents the type of ride. Include the following values: FunRides, MTBRides, SocialRides, and LeagueRides) (2)

```
public enum RideType
{
    ...
}
```

Question 4 – Create the Entry Class [5 Marks]

Create a new public class called Models/Entry.cs which represents all actual ride entries. The class should contain the following items:

1. Create all the following properties with public get and set accessors. Choose the most efficient data type for each property. In addition to the properties, your Entry class should also cater for nullable fields and display friendly names and the correct field lengths. See the table below: (5)

Property	Description	Key	Data Type	Field Length	Nullable
EntryId	The primary key.	PK	Guid		No
RideId	FK Linked to Rides Table	FK	Guid		No
MemberID	FK Linked to ASPNETUsers Id		Int		No
RideTime	The members final time for the event.		Datetime		Yes

Table 3 Entry Class

e.g.

```
namespace Your_Student_ID.Models
{
    public class Entry
    {
        ...
    }
}
```

Question 5 - Scaffold the CRUD Pages [20 Marks]

Now you need to scaffold the new Razor Pages using Entity Framework (CRUD). First we will do the Members Class and then the others.

1. Scaffold the pages for Members
dotnet aspnet-codegenerator razorpage -m Member -dc ApplicationDbContext -outDir Pages/Members-udl --referenceScriptLibraries -dbProvider sqlite-f
2. The Scaffolding should have added the following pages to your project: **(5)**
Added Razor Page : \Pages\Members\Create.cshtml
Added PageModel : \Pages\Members\Create.cshtml.cs
Added Razor Page : \Pages\Members\Edit.cshtml
Added PageModel : \Pages\Members\Edit.cshtml.cs
Added Razor Page : \Pages\Members\Details.cshtml
Added PageModel : \Pages\Members\Details.cshtml.cs
Added Razor Page : \Pages\Members\Delete.cshtml
Added PageModel : \Pages\Members\Delete.cshtml.cs
Added Razor Page : \Pages\Members\Index.cshtml
Added PageModel : \Pages\Members\Index.cshtml.cs
3. Scaffold the Rides and the Entries Classes in the same way as you did the Members Class. **(10)**
4. Add the Navigation links for Members, Rides and Entries to the _Layout.cshtml page.

```
<li class="nav-item">
<a class="nav-link text-dark" asp-area="" asp-page="/Members/Index">Members</a>
</li>
<li class="nav-item">
<a class="nav-link text-dark" asp-area="" asp-page="/Rides/Index">Rides</a>
</li>
<li class="nav-item">
<a class="nav-link text-dark" asp-area="" asp-page="/Entries/Index">Entries</a>
</li>
```

5. Add the following to Program.cs just before app.UseHttpsRedirection();

```
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;

    var context = services.GetRequiredService<ApplicationDbContext>();
}
```
6. Check that the ApplicationDbContext class has been updated with the Member, Ride and Entry classes .
7. Update the Data/ApplicationDbContext class **(3)**
 - a) Because an entity set contains multiple entities, many developers prefer the DbSet property names should be plural. Rename the Member Entity to Members, Ride to Rides and Entry to Entries.

- b) To make the Razor Pages code match the new DbSet name, make a global change from: `_context.Member.` to: `_context.Members.` (rename all occurrences). You can use command + shift + F (mac) or Ctrl + Shift + F (Windows) to search for the files.
- c) Do the same for the Ride and the Entry tables. Add the following code to automatically create the tables when the model is created, just after the line

```
public DbSet<Your_Student_ID.Models.Entry> Entries { get; set; }
```

- d)


```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<Ride>().ToTable("Rides");
    modelBuilder.Entity<Entry>().ToTable("Entries");
    modelBuilder.Entity<Member>().ToTable("Members");
}
```

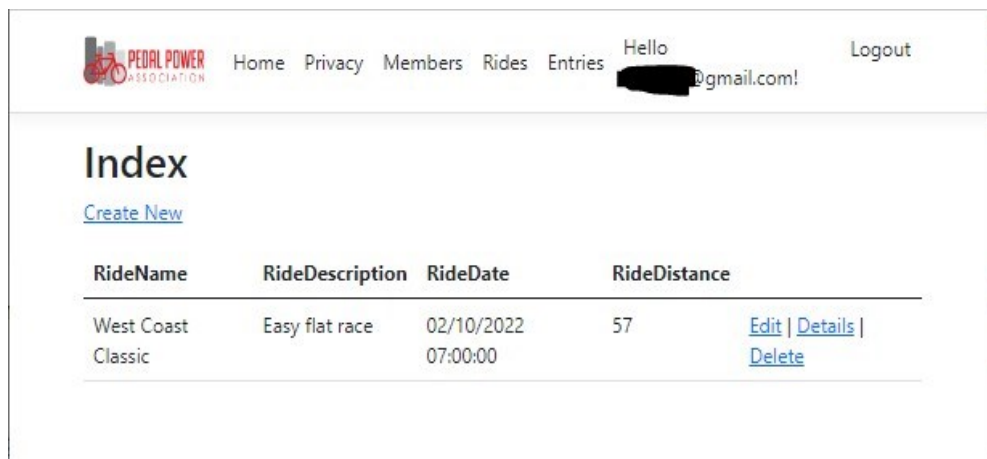
- 8. Add a migration for the custom data and update the database.
- 9. Check your database using the database update output,


```
dotnet ef dbcontext list,
```

```
dotnet ef dbcontext info,
```

```
dotnet ef migrations script
```

 or the database explorer from extensions (e.g. Database Client). You should now have 3 new tables, namely Members, Rides and Entries. **(2)**
- 10. Run your application to check that it works and add a new Member, Ride and Entry. You can use one of the Entries examples from the case study.



Adding a new PPA Ride

In your create Member cshtml file, make sure to add this for a drop down menu of your enumerators when creating members:

```
<div class="form-group">
    <label asp-for="Member.MemberType" class="control-label"></label>
    <select asp-for="Member.MemberType" class="form-control"
        asp-items="Html.GetEnumSelectList<Your_Student_ID.Models.MemberType>()">
    </select>
    <span asp-validation-for="Member.MemberType" class="text-danger"></span>
</div>
```

Question 6 – Creating the User Roles & Security [10 Marks]

You will need to add the following user roles and security functionality:

1. Add Role based authentication in your Program.cs file.

```
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
    options.SignIn.RequireConfirmedAccount = false)
    .AddRoles<IdentityRole>() // Enable roles
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

2. Set the fallback authorization policy to require users to be authenticated.

```
builder.Services.AddAuthorization(options =>
{
    options.FallbackPolicy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
});
```

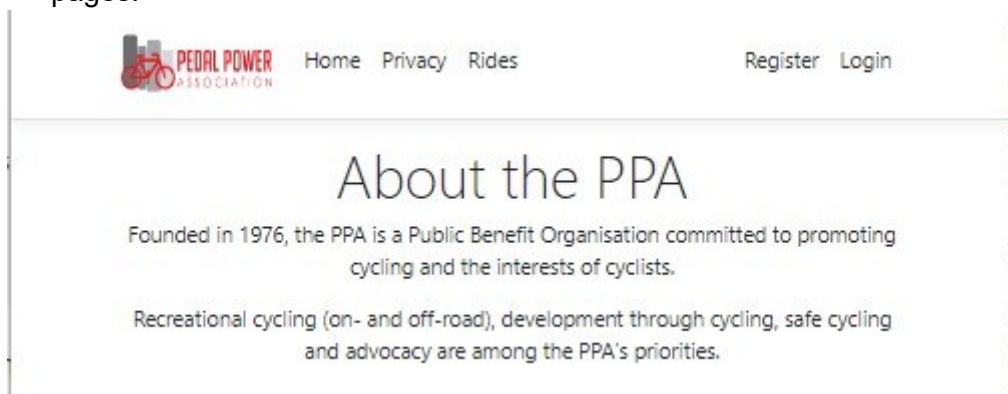
3. Allow [AllowAnonymous] access to the Index, Privacy, Rides, Register, Login pages so that so anonymous users can get information about the site before they register. (2)

E.g.

[AllowAnonymous]

```
public class IndexModel : PageModel
{
    public void OnGet() { }
}
```

4. Update the app to require [Authorization] for the Members info and Entries pages.



Restrict access to the Members and Entries page if the user has not logged in. (2)

5. Update your App to create the following roles/permissions:
 - a) **PPA Administrators** are allowed to view, create, edit Entries, Members and Rides. (2)
 - b) **PPA Members** are allowed to create, read and edit membership and create Rides. (2)
 - c) **PPA Members** can only view their **own** ride entries/times and not edit or delete them. (2)

Note: To simplify testing you can add the first user that is registered on the system as a PPA Administrator.

That's all for now...please upload your entire project to Github.